AMEY CHOPDE

MACHINE LEARNING / APPLIED AI / SOFTWARE

PROFILE AI/ML-focused engineer interested in turning machine-learning methods into practical software systems, from computer vision and medical imaging to language-model applications and developer tools.

Professional overview

My work and project interests sit at the intersection of machine learning, applied AI, and software development. I have explored image classification and object detection, satellite-image analysis, NLP-oriented tasks, local language models, and application interfaces that make models usable beyond notebooks.

I enjoy the full path from problem definition and data preparation through model experimentation, evaluation, and integration. I am particularly interested in building useful, testable systems rather than treating a trained model as the final product.

Technical direction

- Machine learning and deep learning, especially computer vision.
- Generative AI, retrieval-augmented generation, and agentic workflows.
- Python-based applications, APIs, and model integration.
- Exploratory hardware and edge-device projects.

Education

B.Tech — Artificial Intelligence and Machine Learning

G H Raisoni College of Engineering, Nagpur, Maharashtra

Contact & links

Email: ameychopde2004@gmail.com | Phone: 7058369823

Portfolio | GitHub | LinkedIn

Location: Nagpur, Maharashtra, India

Education and technical foundation

Bachelor of Technology — Artificial Intelligence and Machine Learning

G H Raisoni College of Engineering (GHRC), Nagpur, Maharashtra

Academic focus: artificial intelligence, machine learning, programming, and applied computing.

Areas of study and ongoing learning

- Machine learning fundamentals: supervised learning, classification, model evaluation, and practical data preparation.
- Deep learning: neural networks, convolutional neural networks, transfer learning, and computer-vision pipelines.
- Programming with Python for experimentation, automation, and application development.
- Data handling and exploratory analysis using common Python tools.
- Natural language processing and language-model application patterns.
- Software integration: connecting models to interfaces and backend services.

Learning approach

My learning approach is project-driven. I use implementation work to connect theoretical concepts to practical decisions such as data quality, class balance, evaluation metrics, inference workflows, and usability. I also use documentation, small experiments, and iterative debugging to build familiarity with unfamiliar libraries and tools.

Technical practice environment

My development experience has included Python-based environments, Jupyter and Google Colab, local Windows development, WSL/Ubuntu, and local model experimentation. I have also explored running models on consumer hardware while balancing GPU memory, disk space, and available compute.

Professional learning priorities

- Build stronger, reproducible ML pipelines with clear validation and test procedures.
- Improve model deployment and backend integration practices.
- Develop deeper understanding of retrieval, evaluation, and safety for LLM applications.
- Document projects with architecture, metrics, limitations, and instructions for reproduction.

Professional experience

Machine Learning Engineer Intern — Neurons

Part-time internship · 18 months

Worked in a part-time internship capacity with an AI/ML focus. The final public-facing version should include only responsibilities and outcomes that can be substantiated by the work performed.

Experience areas to detail

- Machine-learning experimentation and implementation.
- Data preparation and model evaluation workflows.
- Applied AI/ML project tasks and technical testing.
- Collaboration, progress reporting, and project documentation where applicable.

Machine Learning Engineer — BETiC

Medical imaging / distal-radius fracture analysis project

Contributed to an applied medical-imaging project involving X-ray classification and fracture localization. The known project work includes MobileNetV2-based classification, YOLOv8-based localization, and an offline application concept integrating model inference.

- Worked with a dataset of 550 X-ray images across normal, intra-articular, and extra-articular categories.
- Explored a MobileNetV2 classification pipeline and reported test accuracy of 96.55% in the project notes.
- Worked with YOLOv8 localization results reported as mAP@0.5 88.26, precision 87.76, and recall 83.08.
- Supported an offline application workflow using Flask and Tkinter.

Machine Learning Engineer — Pragmatyc

AI/ML engineering experience

Role title supplied by the candidate. Specific projects, responsibilities, dates, and measurable outcomes have not yet been provided; keep this entry concise until those details are confirmed.

Machine learning and computer vision

My applied ML interests include building and evaluating models for image-based classification, localization, and pattern recognition. My project work has exposed me to data preparation, transfer learning, evaluation metrics, and the practical step of wrapping inference in a usable application.

Machine-learning workflow

- Problem framing: define the prediction task, input format, target labels, and success criteria.
- Data preparation: inspect data, organize labels, check class distributions, and prepare training/validation/test splits.
- Model development: establish a baseline, select an appropriate architecture, and run controlled experiments.
- Evaluation: interpret accuracy and class-sensitive metrics such as precision, recall, and mean average precision.
- Integration: expose inference through a simple application or backend where appropriate.
- Documentation: record data sources, assumptions, model limitations, and reproducibility steps.

Known model and library exposure

Area	Tools / methods
Image classification	CNNs, transfer learning, MobileNetV2
Object detection	YOLOv8; precision, recall, mAP evaluation
Data processing	Python, NumPy, pandas, image preprocessing
Experimentation	Jupyter Notebook, Google Colab
Application integration	Flask, Tkinter

Evaluation mindset

A single headline metric does not fully describe a model. Dataset size, class balance, split strategy, leakage risk, threshold selection, and the difference between offline metrics and real-world performance all matter. Medical and safety-sensitive applications also require careful limits on what a model result can be used to conclude.

Medical imaging and accessible communication

Project 1 — Distal-radius fracture analysis

Domain: medical imaging · Computer vision · Offline inference

Objective: explore automated support for identifying fracture-related patterns in wrist X-ray images and localizing relevant regions.

Dataset and methods

- Dataset notes describe 550 X-ray images: 225 normal, 112 intra-articular, and 113 extra-articular.
- MobileNetV2 was used for image classification; project notes report 96.55% test accuracy.
- YOLOv8 was used for localization; recorded results include mAP@0.5 of 88.26, precision of 87.76, and recall of 83.08.
- A Flask and Tkinter-based offline application workflow was explored to make inference accessible through a user interface.

Engineering considerations

Important considerations include patient-data governance, image quality, class imbalance, separation of patient-level data between splits, explainability, and the need for clinical review. The model should be described as a research or decision-support prototype, not as a replacement for professional diagnosis.

Project 2 — SIGNLYF

Domain: accessibility · Sign-language interfaces · Speech processing

Concept: a communication platform intended to connect sign-language input and spoken-language output with the reverse speech-to-sign pathway.

- ASL-to-speech and speech-to-ASL are treated as distinct model and interface pipelines.
- The planned integration brings model outputs and animation into a shared application experience.
- The work has included exploring a PWA-first approach, backend integration, a local database/RAG concept, and an animation engine built with Pygame.

Integration plan

The proposed system separates the front end, inference services, and sign-animation engine. A common backend can coordinate requests and normalize responses, while the interface handles recording or input selection, results, and animation playback.

Research and applied machine learning

Project 3 — Early pre-cyclonic cloud pattern recognition

Domain: satellite imagery · Weather-pattern exploration · Deep learning

Objective: explore whether satellite image patterns can support early recognition of cloud formations associated with pre-cyclonic conditions.

- Project notes describe GOES-18 ABI-L2-MCMIPC data from 2024/200.
- The prepared dataset notes describe 432 images derived from 108 time points across four channels: C01, C02, C13, and C14.
- The pipeline converts NetCDF source data into image inputs for a web application.
- A CNN-based approach with MobileNetV2 was explored; the described split was 80% training and 20% testing.

Key engineering questions include temporal separation between training and test samples, geographic and seasonal generalization, consistent preprocessing, and avoiding overclaims about prediction lead time. This should be presented as exploratory research unless validated against a suitable operational baseline.

Project 4 — Indian cyber-scam phone-call dataset exploration

Domain: NLP / text classification · Dataset preparation

- Explored the dataset ysangam/Indian_Cyber_Scam_PhoneCall_Hinglish_Dataset.
- Project notes record a reduction from 10,000 initial records to 743 after cleaning.
- The cleaned labels were recorded as 633 examples for label 1 and 110 for label 0, indicating substantial class imbalance.

A strong project write-up should explain the cleaning rules, label meaning, language variation, train/test strategy, class-sensitive metrics, and how the model would be evaluated on unseen scam patterns.

Project 5 — Telecom churn prediction

Domain: tabular machine learning · Classification

A Colab-based telecom churn exercise used Logistic Regression with five-fold cross-validation. Project notes record a mean score of 0.836; the precise metric should be verified before publication.

Research habits

For each experiment, document the dataset source, preprocessing, model configuration, split strategy, baseline, evaluation metric, known limitations, and reproducibility instructions. This makes results easier to interpret and compare.

Generative AI, NLP and agentic systems

I am interested in language-model applications that combine natural-language interaction with retrieval, tools, memory, and practical software workflows. My exploration includes local model execution and agent frameworks as well as designs for systems that distinguish recommendations from actions.

RAG and retrieval-based applications

- Explore retrieval-augmented generation (RAG) to ground responses in a selected knowledge source.
- Investigate document ingestion, chunking, embeddings, retrieval, prompt construction, and answer evaluation.
- Explore AutoRAG and Self-RAG concepts as approaches to retrieval and response-quality improvement.
- Treat citation quality, retrieval relevance, hallucination checks, and data freshness as first-class evaluation concerns.

LangChain and tool-enabled workflows

Learning focus: composing model calls, tools, agent-style workflows, and persistent context. A reliable implementation should validate tool inputs, handle failures, maintain logs, and require explicit user approval before consequential actions.

MIND-OS — Metacognitive Intelligent Neural Design

Concept: a cross-platform personal AI operations assistant designed to help organize tasks, calendar entries, reminders, searchable notes, and project context.

- Natural-language requests mapped to explicit tools and structured actions.
- Persistent memory for useful user-approved context.
- Human approval before important external actions or irreversible changes.
- Searchable notes and context around classes, project deadlines, and study goals.
- A separation between suggesting an action and executing it.

Local LLM experimentation

Explored local inference with Ollama and interest in running larger open models. Hardware constraints such as GPU VRAM, system RAM, disk space, quantization, context length, and inference speed influence which models are practical on a local machine.

Software engineering and hardware exploration

Programming and application development

- Python for machine learning experiments, data preparation, and automation.
- Flask for lightweight web/API integration and Tkinter for desktop interfaces.
- OpenCV and MediaPipe exposure through a virtual-mouse computer-vision project.
- Jupyter Notebook and Google Colab for experimentation and sharing reproducible workflows.
- Web and app development interests, including front-end/back-end integration and PWA concepts.
- Git-based repository workflows and environment management where used.

Development environments

Work environments have included Windows 11, WSL/Ubuntu, Python environments, local GPU experimentation, and Google Colab. Experience with both local and hosted environments helps surface practical issues such as package compatibility, compute limits, path handling, and dependency reproducibility.

Embedded and edge-device interests

Hardware exploration includes ESP32 development boards, an ESP32-C3, Raspberry Pi Zero 2 W, ESP32-CAM, Arduino, OLED displays, and rotary encoders. These platforms support interest in compact interfaces, edge devices, and connecting software with physical inputs and outputs.

Example hardware concepts

- A compact, Walkman-style audio player with SD-card storage and a wired headphone output.
- Small display-and-control interfaces using an OLED and rotary encoder.
- Experiments combining local AI software with inexpensive embedded devices.

Engineering practices to demonstrate

- Clear repository structure and setup instructions.
- Environment files and pinned dependencies where appropriate.
- Input validation, useful error messages, and logging.
- Unit or integration tests for critical components.
- Separation of model logic, service/API logic, and interface code.

Selected project catalogue

This catalogue gives the portfolio website a structured overview. Each project page can later include a repository link, screenshots, architecture diagram, demo, and a concise explanation of individual contributions.

Distal-radius fracture analysis

Medical imaging / computer vision | MobileNetV2 classification; YOLOv8 localization; Flask/Tkinter offline workflow.

Status: Research prototype / verify deployment status

SIGNLYF

Accessibility / speech and sign language | ASL-to-speech and speech-to-ASL integration concept; PWA and animation-engine coordination.

Status: Ongoing integration

Early pre-cyclonic cloud recognition

Satellite imagery / deep learning | GOES-18 data preparation, NetCDF-to-image workflow, CNN/MobileNetV2 exploration.

Status: Research project

Cyber-scam call text classification

NLP / classification | Hinglish dataset cleaning, label inspection, class imbalance analysis.

Status: Exploration

Telecom churn prediction

Tabular ML | Logistic Regression and five-fold cross-validation; recorded mean score 0.836, metric to verify.

Status: Experiment

Virtual mouse

Computer vision / interaction | Hand-tracking interaction concept using OpenCV/MediaPipe.

Status: Project experiment

MIND-OS

Agentic AI / productivity | Tool calling, persistent memory, searchable notes, approval before important actions.

Status: Concept / learning project

Local LLM experiments

LLM infrastructure | Ollama-based local inference and model/hardware trade-off exploration.

Status: Ongoing experimentation

Recommended project-page template

- Problem statement and intended user.
- My individual contribution and team responsibilities.
- Architecture and data flow.
- Dataset source, preprocessing, and evaluation methodology.
- Results, limitations, and lessons learned.
- Repository, demo, screenshots, and setup instructions.

Technical skills and professional direction

Skills inventory

Programming	Python
Machine learning	Classification, model evaluation, data preprocessing, cross-validation
Deep learning	CNNs, transfer learning, MobileNetV2
Computer vision	Image classification, object detection, OpenCV, YOLOv8
Generative AI / NLP	RAG concepts, LangChain learning, local LLM experimentation, text classification
Application development	Flask, Tkinter, PWA concepts, API integration
Data / notebooks	NumPy, pandas, Jupyter Notebook, Google Colab
Systems / hardware	Windows, WSL/Ubuntu, Ollama, ESP32 family, Raspberry Pi Zero 2 W

Professional interests

- Applied machine learning and computer vision.
- Generative AI, retrieval systems, and agentic applications.
- AI-enabled web and application development.
- Developer tools, local inference, and edge computing.
- Game development as an area of interest.

Professional direction

My goal is to keep developing as an engineer who can connect machine-learning models with usable software. I am particularly interested in systems that solve concrete problems, have measurable behavior, and can be maintained and evaluated after the first prototype.

Portfolio publishing checklist

- Add professional email, phone, portfolio URL, GitHub, and LinkedIn.
- Verify company names, role titles, employment arrangements, and project dates.
- Confirm all metrics against experiment logs or reports.
- Add repository and demo links only when they are accessible and ready to share.
- Mark every project as completed, ongoing, experimental, or planned.
- Review confidentiality and permissions before publishing any workplace or medical-project material.

CLOSING STATEMENT I build and explore AI/ML systems with an emphasis on practical implementation, experimentation, and software integration. My portfolio is a record of the problems I have worked on, the tools I have explored, and the engineering direction I am continuing to develop.